

We see that there are some features that we would be considering for the machine learning model. Some of the actual features that we would be considering are as follows:-

1-age

2-sex

3-chest pain type (4 values)

4-resting blood pressure

5-serum cholestoral in mg/dl

6-fasting blood sugar > 120 mg/dl

7-resting electrocardiographic results (values 0,1,2)

8-maximum heart rate achieved

9-exercise induced angina

10-oldpeak = ST depression induced by exercise relative to rest

11-the slope of the peak exercise ST segment

12-number of major vessels (0-3) colored by flourosopy

13-thal: 3 = normal; 6 = fixed defect; 7 = reversable defect

We would be limiting ourselves to work with small dataset so that we gain an understanding of the overall workflow of machine learning. Once we gain a good understanding of the workflow of machine learning, we can explore other datasets and follow the same procedure for datasets that are large as the process stays the same.

```
In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns #This is used for data visualization
#Below are the Libraries for evaluation of classification machine Learning model
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_sc
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.preprocessing import Normalizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier
```

Read The data

```
In [5]: df= pd.read_csv(r"C:\Users\chitt\Downloads\heart.csv")
df.head()
```

Out[5]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2

Getting the information

In [8]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         303 non-null    int64
1   sex         303 non-null    int64
2   cp          303 non-null    int64
3   trestbps    303 non-null    int64
4   chol        303 non-null    int64
5   fbs         303 non-null    int64
6   restecg     303 non-null    int64
7   thalach     303 non-null    int64
8   exang       303 non-null    int64
9   oldpeak     303 non-null    float64
10  slope       303 non-null    int64
11  ca          303 non-null    int64
12  thal        303 non-null    int64
13  target      303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

Understanding how the data is spread

****In addition to the above operation performed, we also have to see how the values are spread out in the form of mathematical vectors to get a solid understanding of the data. We have to observe the mean and standard deviation (std). Moreover, we also must get to know the minimum and maximum values for every feature that we are considering at this point. Therefore, we would be using describe() which would give us all the above values for all the features that we have considered. I just used 'T' after describe just to make it easy for us to read the table and get to know the features well. It is not mandatory to use 'T' after describe(). We see that there are a few categorical features that are represented as mathematical vectors rather than having those categories. It is due to the fact that machine learning algorithms does not process the information in the form of text. We have to always ensure that when we are giving the input to machine learning algorithms, we have to convert**

all the features in the form of mathematical vectors. Some of the interesting features present in the dataset are age and cholesterol. In general, we see that those who are old and have high cholesterol levels have a higher chance of getting heart diseases. It need not be true always but for the most part, that is what is assumed. Therefore, we can say that just by looking at the features and the values present in them, we might be able to understand how the machine learning algorithms would lay their emphasis on these features.**

In [13]: `df.describe()`

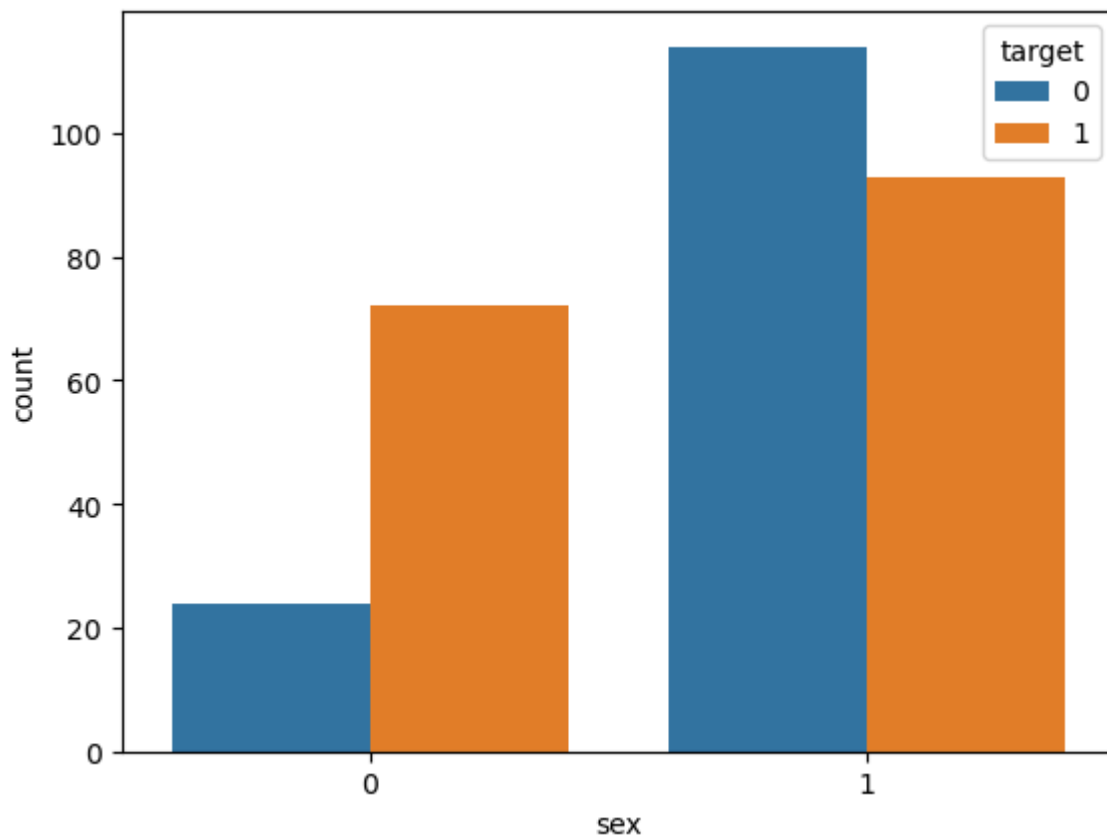
Out[13]:

	age	sex	cp	trestbps	chol	fbs	restecg
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000
mean	54.366337	0.683168	0.966997	131.623762	246.264026	0.148515	0.528000
std	9.082101	0.466011	1.032052	17.538143	51.830751	0.356198	0.525000
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000
25%	47.500000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000
50%	55.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000
75%	61.000000	1.000000	2.000000	140.000000	274.500000	0.000000	1.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000

Countplot for feature 'sex'

In [18]: `sns.countplot(data = df, x = 'sex', hue = 'target')`

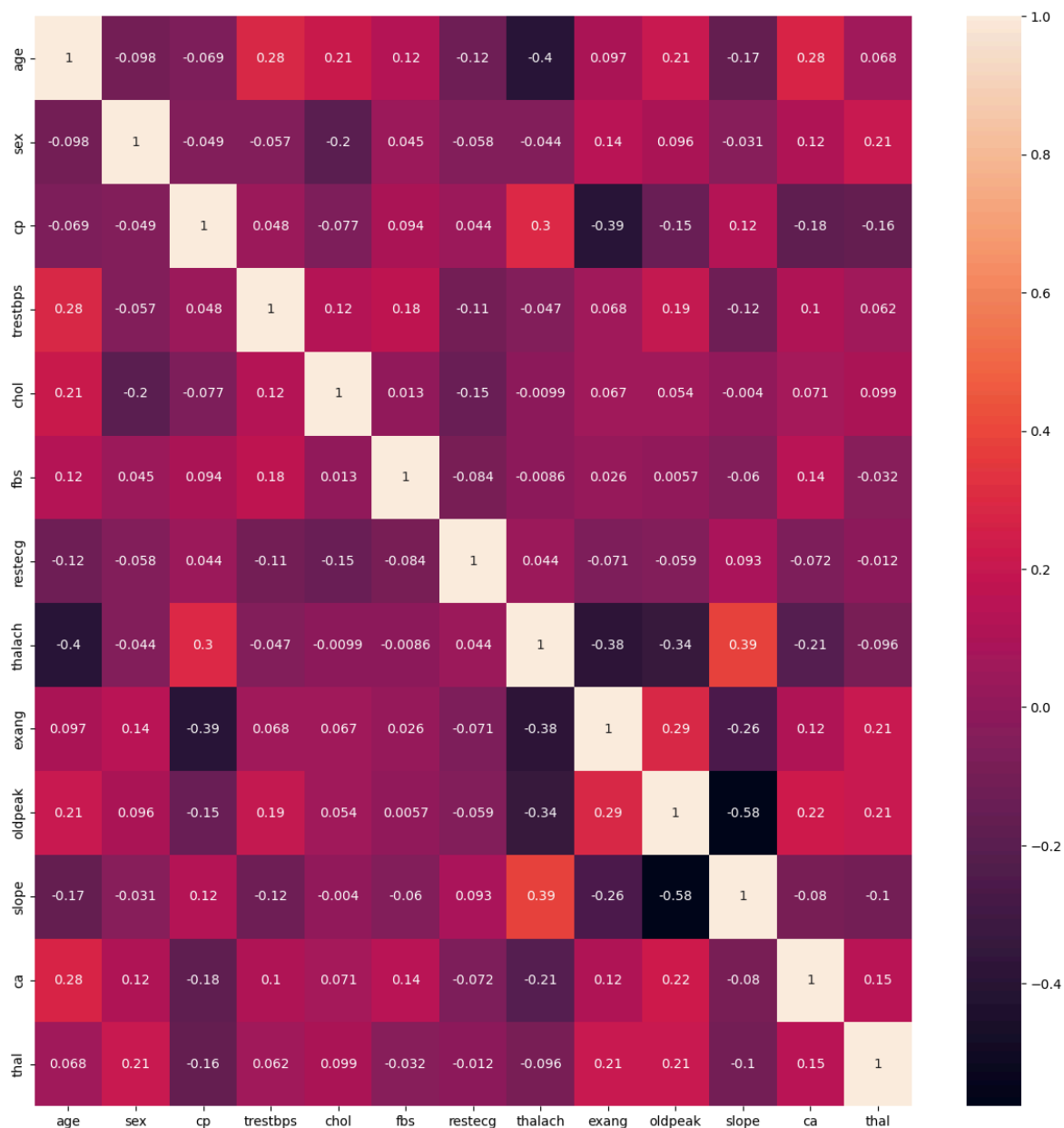
Out[18]: `<Axes: xlabel='sex', ylabel='count'>`



Heatmap to understand correlation

```
In [23]: plt.figure(figsize = (15, 15))  
sns.heatmap(df.drop(['target'], axis = 1).corr(), annot = True)
```

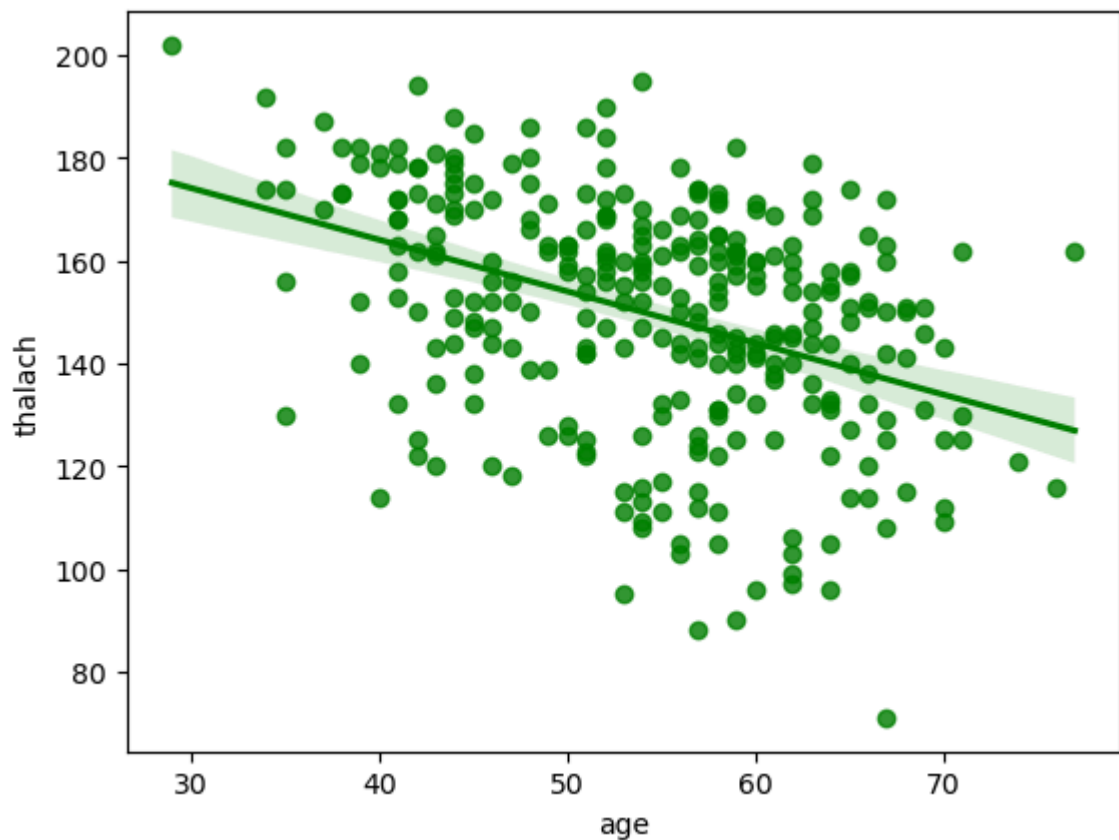
```
Out[23]: <Axes: >
```



Regression plot for features

```
In [29]: sns.regplot(x='age', y='thalach', data = df, color = 'green')
```

```
Out[29]: <Axes: xlabel='age', ylabel='thalach'>
```



In [33]: `df.head()`

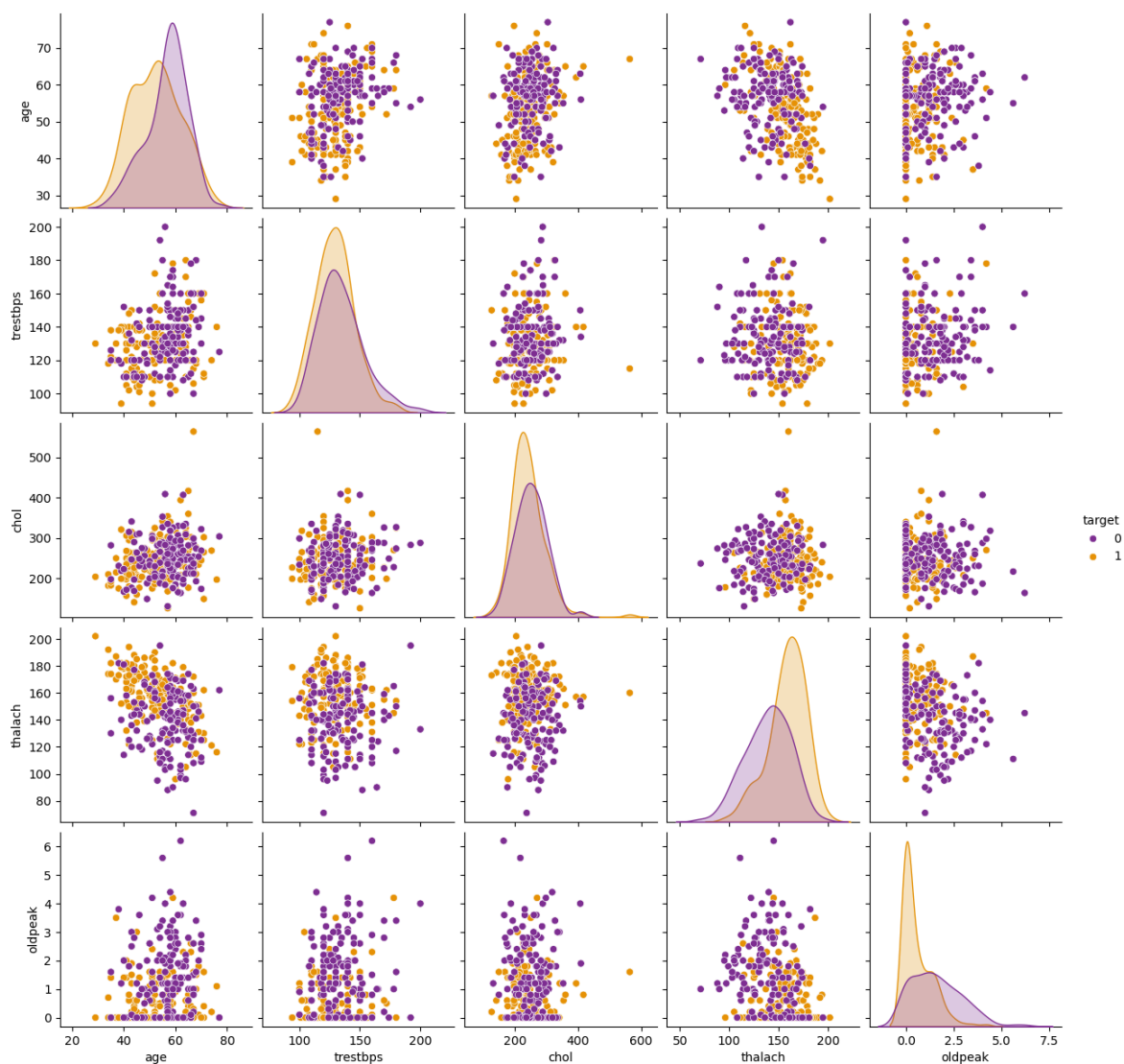
Out[33]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2

Pairplot for the data points

In [36]: `sns.pairplot(df[['age', 'trestbps', 'chol', 'thalach', 'oldpeak', 'target']], hu`

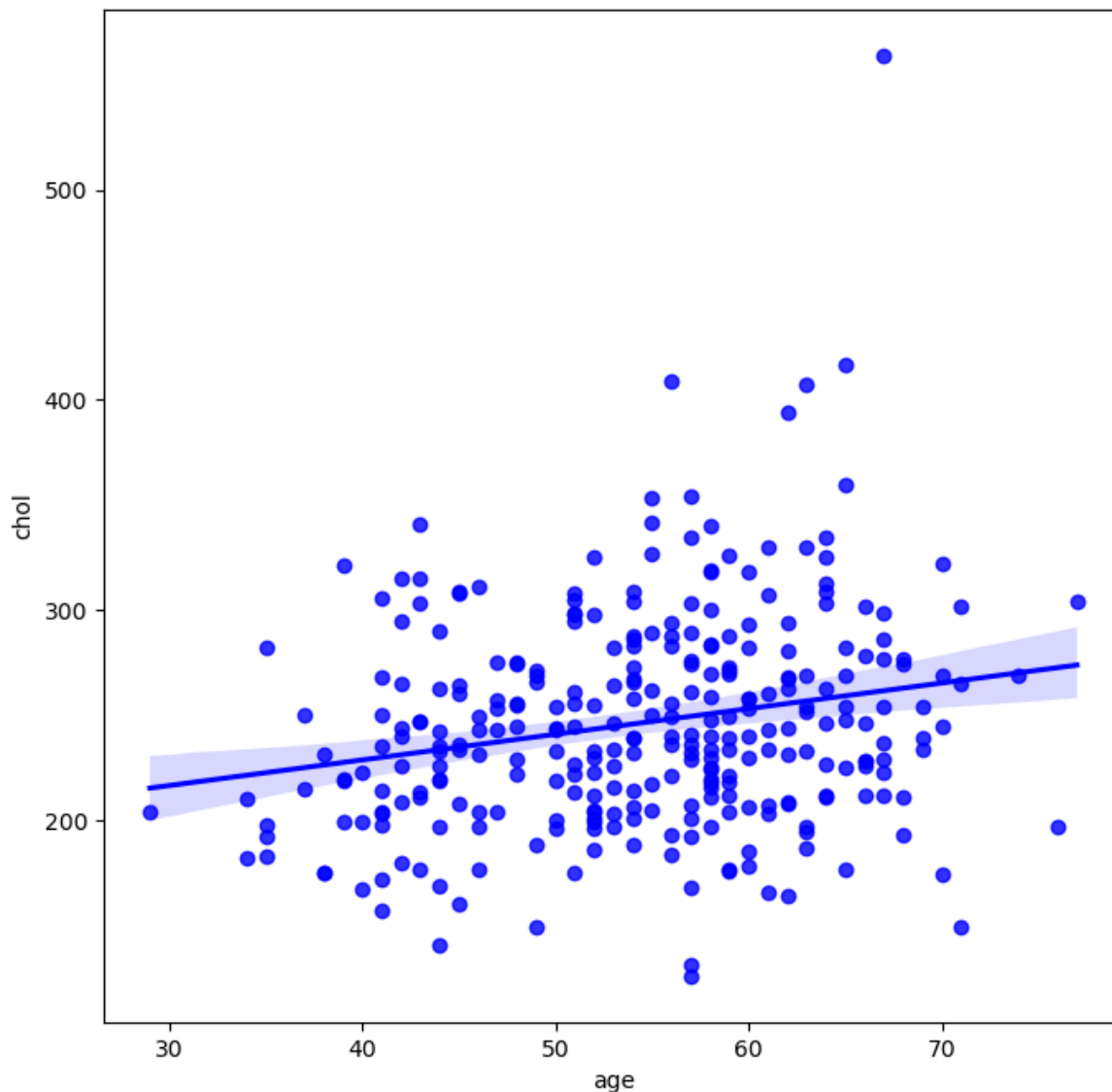
Out[36]: `<seaborn.axisgrid.PairGrid at 0x2a7476c71d0>`



Regression plot for features

```
In [39]: plt.figure(figsize = (8,8))
sns.regplot(data = df, x = 'age', y = 'chol', color='blue')
```

```
Out[39]: <Axes: xlabel='age', ylabel='chol'>
```



In [41]: `df.head()`

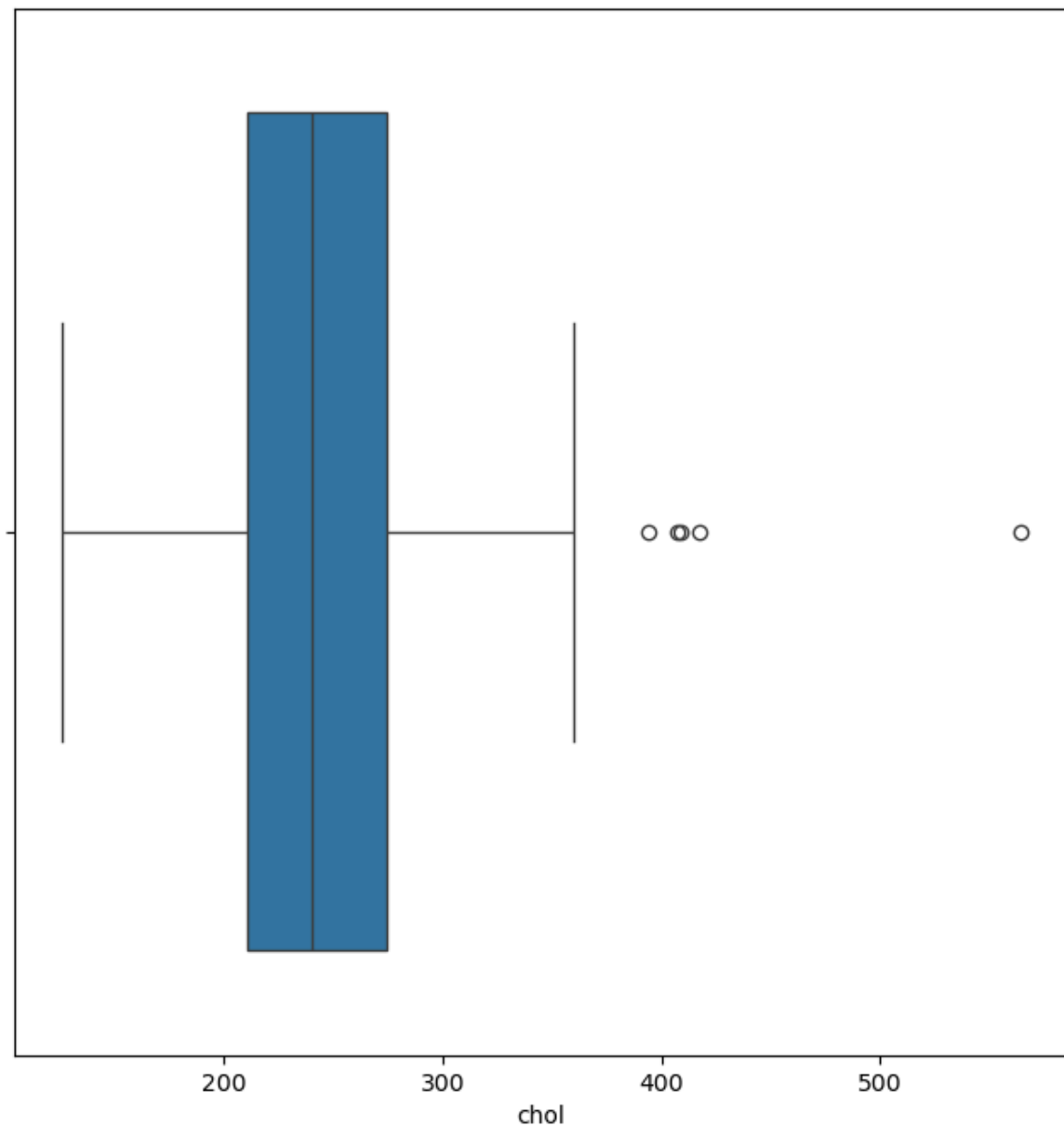
Out[41]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2

Boxplot for feature 'Cholestrol'

In [44]: `plt.figure(figsize= (8,8))`
`sns.boxplot(data =df, x= 'chol')`

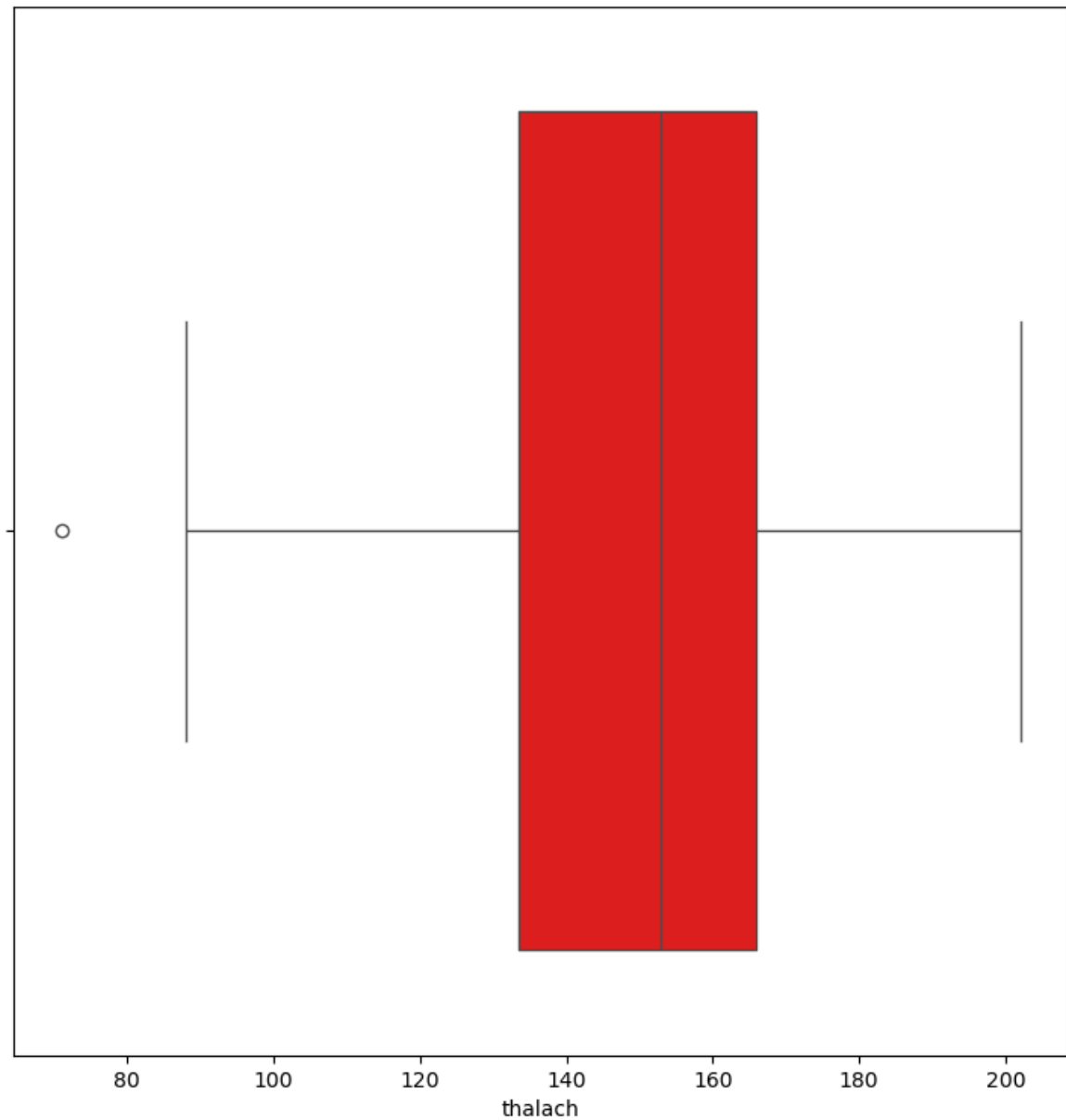
Out[44]: `<Axes: xlabel='chol'>`



Boxplot for 'Thalach'

```
In [50]: plt.figure(figsize=(9,9))  
sns.boxplot(data=df, x='thalach', color='red')
```

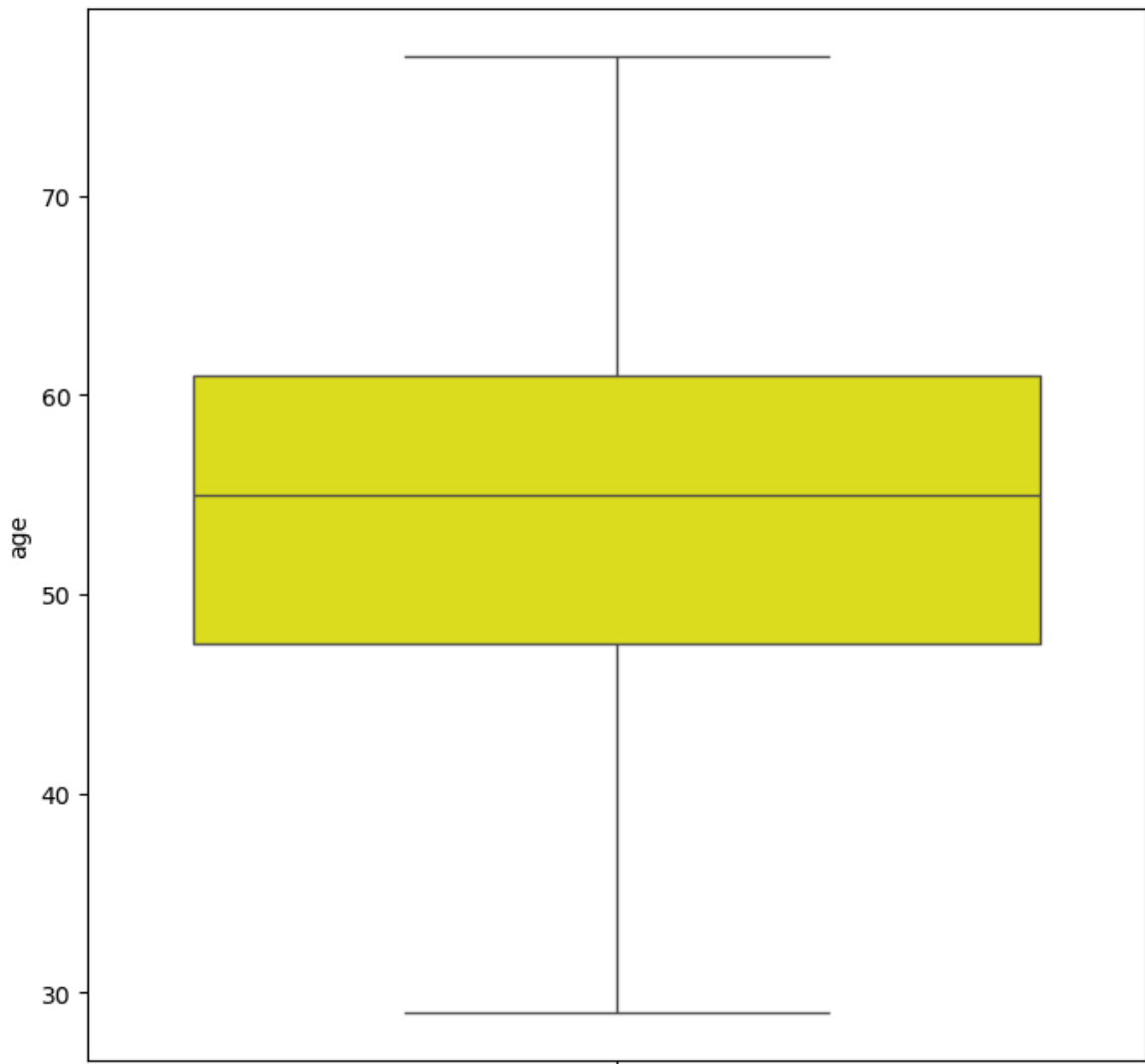
```
Out[50]: <Axes: xlabel='thalach'>
```



Boxplot for feature 'Age'

```
In [55]: plt.figure(figsize= (8,8))  
sns.boxplot(data=df, y='age', color='yellow')
```

```
Out[55]: <Axes: ylabel='age'>
```



In [57]: `df.head()`

Out[57]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2

Assigning value from dataset

In [60]: `X = df.drop(['target'], axis=1)`
`y = df['target']`

Dividing data using train_test_split

```
In [63]: X_train, X_test, y_train, y_test=train_test_split(X,y, test_size=0.3, random_sta
```

Getting the shape

```
In [66]: X_train.shape      #gives the shape of the data under consideration
```

```
Out[66]: (212, 13)
```

```
In [68]: X_test.shape
```

```
Out[68]: (91, 13)
```

Creating Empty List

```
In [71]: accuracy_scaled = []      #This creates an empty list which would be late
f1_score_scaled = []
precision_score_scaled = []
recall_score_scaled = []
roc_auc_score_scaled = []
log_loss_scaled = []

accuracy_normalized = []
f1_score_normalized = []
precision_score_normalized = []
recall_score_normalized = []
roc_auc_score_normalized = []
log_loss_normalized = []
```

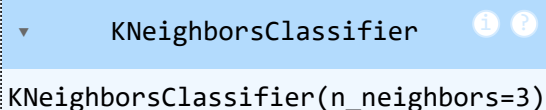
Standarization and Normalization

```
In [74]: scaler = StandardScaler()      #Creating an instance of
scaler.fit(X_train)      #Fitting the input train
X_train_scaled = scaler.transform(X_train)      #Transforming the values
X_test_scaled = scaler.transform(X_test)      #Transforming the values
```

```
In [76]: normalizer = Normalizer()      #Creating an instance of
normalizer.fit(X_train)      #Fitting the input train
X_train_normalized = normalizer.transform(X_train)      #Transforming the values
X_test_normalized = normalizer.transform(X_test)      #Transforming the values
```

K Nearest Neighbors(KNN)

```
In [79]: neigh1 = KNeighborsClassifier(n_neighbors = 3)      #Creating an instance of
neigh1.fit(X_train_scaled, y_train)      #Fitting the model with
neigh2 = KNeighborsClassifier(n_neighbors = 3)      #Creating an instance of
neigh2.fit(X_train_normalized, y_train)      #Fitting the model with
```

```
Out[79]: 
KNeighborsClassifier(n_neighbors=3)
```

```
In [81]: y_test_predict_scaled = neigh1.predict(X_test_scaled) #Getting
y_test_predict_normalized = neigh2.predict(X_test_normalized)
```

Appending data to lists

****In the first cell, we have used various libraries and metrics from sklearn which would give us an estimate of how well our models did on the test set. We have created empty lists before and now we would be using those and appending the results to them. We have some metrics such as accuracy score, precision score and recall score for KNN algorithm. We would now be appending the values to the lists so that they could be later visualized.****

```
In [84]: accuracy_scaled.append(accuracy_score(y_test_predict_scaled, y_test))
f1_score_scaled.append(f1_score(y_test_predict_scaled, y_test))
precision_score_scaled.append(precision_score(y_test_predict_scaled, y_test))
recall_score_scaled.append(recall_score(y_test_predict_scaled, y_test))
roc_auc_score_scaled.append(roc_auc_score(y_test_predict_scaled, y_test))
log_loss_scaled.append(log_loss(y_test_predict_scaled, y_test))

accuracy_normalized.append(accuracy_score(y_test_predict_normalized, y_test))
f1_score_normalized.append(f1_score(y_test_predict_normalized, y_test))
precision_score_normalized.append(precision_score(y_test_predict_normalized, y_t
recall_score_normalized.append(recall_score(y_test_predict_normalized, y_test))
roc_auc_score_normalized.append(roc_auc_score(y_test_predict_normalized, y_test)
log_loss_normalized.append(log_loss(y_test_predict_normalized, y_test))
```

Logestic Regression

```
In [87]: clf1 = LogisticRegression(random_state = 100)
clf2 = LogisticRegression(random_state = 100)
clf1.fit(X_train_scaled, y_train)
clf2.fit(X_train_normalized, y_train)
y_test_predict_scaled = clf1.predict(X_test_scaled)
y_test_predict_normalized = clf2.predict(X_test_normalized)
```

Appending data to lists

****We would now be storing the results of the metrics as we did in the previous cells. We have created those lists before and we would just append these values to them so that it becomes easier for us to visualize the plots when needed and gain a solid understanding of the machine learning solutions.****

```
accuracy_scaled.append(accuracy_score(y_test_predict_scaled, y_test))
f1_score_scaled.append(f1_score(y_test_predict_scaled, y_test))
precision_score_scaled.append(precision_score(y_test_predict_scaled, y_test))
recall_score_scaled.append(recall_score(y_test_predict_scaled, y_test))
roc_auc_score_scaled.append(roc_auc_score(y_test_predict_scaled, y_test))
log_loss_scaled.append(log_loss(y_test_predict_scaled, y_test))
```

```

accuracy_normalized.append(accuracy_score(y_test_predict_normalized, y_test))
f1_score_normalized.append(f1_score(y_test_predict_normalized, y_test))
precision_score_normalized.append(precision_score(y_test_predict_normalized, y_test))
recall_score_normalized.append(recall_score(y_test_predict_normalized, y_test))
roc_auc_score_normalized.append(roc_auc_score(y_test_predict_normalized, y_test))
log_loss_normalized.append(log_loss(y_test_predict_normalized, y_test))

```

Naive Bayes Model

```

In [93]: clf1 = GaussianNB()
         clf2 = GaussianNB()
         clf1.fit(X_train_scaled, y_train)
         clf2.fit(X_train_normalized, y_train)
         y_test_predict_scaled = clf1.predict(X_test_scaled)
         y_test_predict_normalized = clf2.predict(X_test_normalized)

```

Appending data to lists

We would append the values just like we did in the previous cells to the lists that we have created before. We would continuously append these values as they would be needed later for visualization.

```

In [99]: accuracy_scaled.append(accuracy_score(y_test_predict_scaled, y_test))
         f1_score_scaled.append(f1_score(y_test_predict_scaled, y_test))
         precision_score_scaled.append(precision_score(y_test_predict_scaled, y_test))
         recall_score_scaled.append(recall_score(y_test_predict_scaled, y_test))
         roc_auc_score_scaled.append(roc_auc_score(y_test_predict_scaled, y_test))
         log_loss_scaled.append(log_loss(y_test_predict_scaled, y_test))

         accuracy_normalized.append(accuracy_score(y_test_predict_normalized, y_test))
         f1_score_normalized.append(f1_score(y_test_predict_normalized, y_test))
         precision_score_normalized.append(precision_score(y_test_predict_normalized, y_t
         recall_score_normalized.append(recall_score(y_test_predict_normalized, y_test))
         roc_auc_score_normalized.append(roc_auc_score(y_test_predict_normalized, y_test)
         log_loss_normalized.append(log_loss(y_test_predict_normalized, y_test))

```

Random Forest Classifier

```

In [102...: clf1 = RandomForestClassifier(max_depth = 10, random_state = 100)
            clf2 = RandomForestClassifier(max_depth = 10, random_state = 100)
            clf1.fit(X_train_scaled, y_train)
            clf2.fit(X_train_normalized, y_train)
            y_test_predict_scaled = clf1.predict(X_test_scaled)
            y_test_predict_normalized = clf2.predict(X_test_normalized)

```

Appending data to lists

****We would append the values of the random forest results metrics just as we did for the previous models. We would store the values in the lists that we have created before and use them later for data visualizations.****

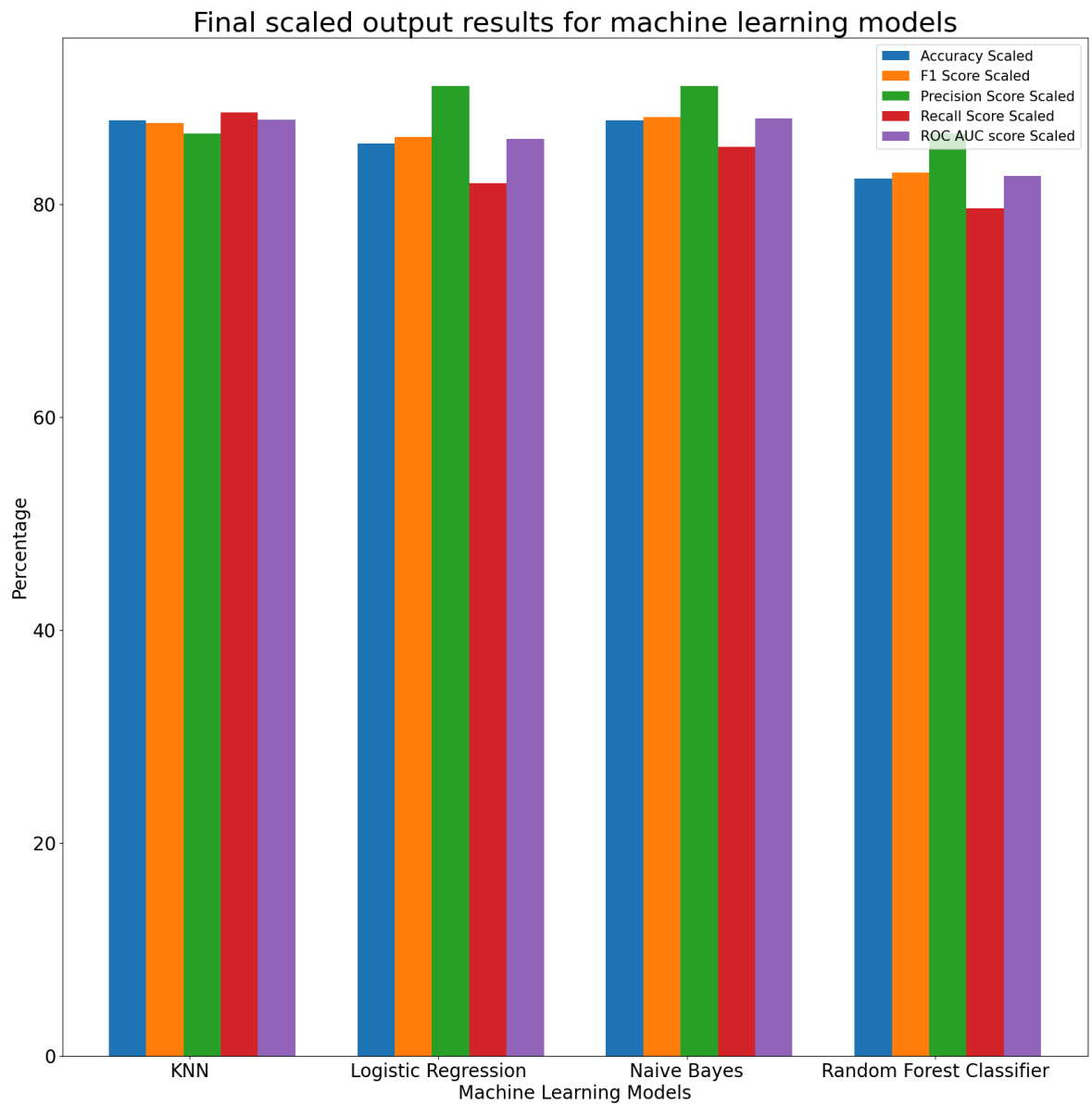
```
In [105... accuracy_scaled.append(accuracy_score(y_test_predict_scaled, y_test))
f1_score_scaled.append(f1_score(y_test_predict_scaled, y_test))
precision_score_scaled.append(precision_score(y_test_predict_scaled, y_test))
recall_score_scaled.append(recall_score(y_test_predict_scaled, y_test))
roc_auc_score_scaled.append(roc_auc_score(y_test_predict_scaled, y_test))
log_loss_scaled.append(log_loss(y_test_predict_scaled, y_test))

accuracy_normalized.append(accuracy_score(y_test_predict_normalized, y_test))
f1_score_normalized.append(f1_score(y_test_predict_normalized, y_test))
precision_score_normalized.append(precision_score(y_test_predict_normalized, y_test))
recall_score_normalized.append(recall_score(y_test_predict_normalized, y_test))
roc_auc_score_normalized.append(roc_auc_score(y_test_predict_normalized, y_test))
log_loss_normalized.append(log_loss(y_test_predict_normalized, y_test))
```

Barplot for metrics with standardized features

```
In [108... # knn, logisticregression, naivebayes, randomforestclassifier model values are p
models = ['KNN', 'Logistic Regression', 'Naive Bayes', 'Random Forest Classifier']
models = np.arange(len(models))
plt.figure(figsize = (20, 20)) #Increasing the size of the figure so t
plt.yticks(fontsize = 20) #Increasing the fontsize of the y axis
#getting a barplot between models and accuracy_scaled list and multiplying those
plt.bar(models, [i * 100 for i in accuracy_scaled], width = 0.15)
#Performing the same operations for other lists so that we would draw them later
plt.bar(models + 0.15, [i * 100 for i in f1_score_scaled], width = 0.15)
plt.bar(models + 0.15 * 2, [i * 100 for i in precision_score_scaled], width = 0.15)
plt.bar(models + 0.15 * 3, [i * 100 for i in recall_score_scaled], width = 0.15)
plt.bar(models + 0.15 * 4, [i * 100 for i in roc_auc_score_scaled], width = 0.15)
plt.legend(['Accuracy Scaled', 'F1 Score Scaled', 'Precision Score Scaled', 'Recall Score Scaled', 'ROC AUC Score Scaled'])
plt.xticks([i + 0.25 for i in range(4)], ['KNN', 'Logistic Regression', 'Naive Bayes', 'Random Forest Classifier'])
plt.xlabel('Machine Learning Models', fontsize = 20) #Creating a label for the x-axis
plt.ylabel('Percentage', fontsize = 20) #Creating a label for the y-axis
plt.title('Final scaled output results for machine learning models', fontsize = 20)
```

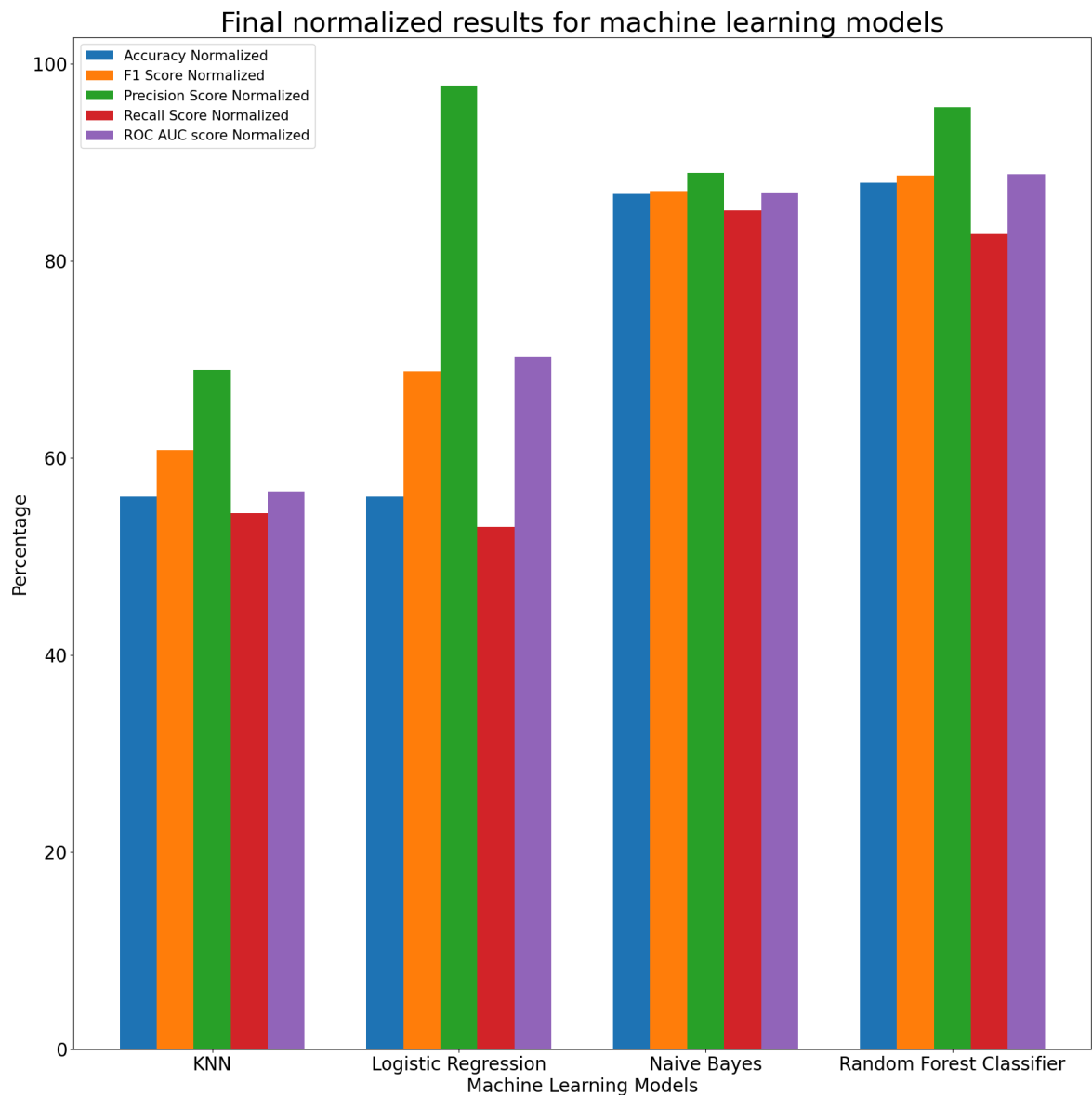
```
Out[108... Text(0.5, 1.0, 'Final scaled output results for machine learning models')
```



Barplot for metrics with normlized features

```
In [111...] # knn, logisticregression, naivebayes, randomforestclassifier model values are p
models = ['KNN', 'Logistic Regression', 'Naive Bayes', 'Random Forest Classifier']
models = np.arange(len(models))
plt.figure(figsize = (20, 20))
plt.yticks(fontsize = 20)
plt.bar(models, [i * 100 for i in accuracy_normalized], width = 0.15)
plt.bar(models + 0.15, [i * 100 for i in f1_score_normalized], width = 0.15)
plt.bar(models + 0.15 * 2, [i * 100 for i in precision_score_normalized], width = 0.15)
plt.bar(models + 0.15 * 3, [i * 100 for i in recall_score_normalized], width = 0.15)
plt.bar(models + 0.15 * 4, [i * 100 for i in roc_auc_score_normalized], width = 0.15)
plt.legend(['Accuracy Normalized', 'F1 Score Normalized', 'Precision Score Normalized', 'Recall Score Normalized', 'ROC AUC score Normalized'])
plt.xticks([i + 0.3 for i in range(4)], ['KNN', 'Logistic Regression', 'Naive Bayes', 'Random Forest Classifier'])
plt.xlabel('Machine Learning Models', fontsize = 20)
plt.ylabel('Percentage', fontsize = 20)
plt.title("Final normalized results for machine learning models", fontsize = 30)
```

```
Out[111...] Text(0.5, 1.0, 'Final normalized results for machine learning models')
```

Conclusion

****1.We've learned to use the various machine learning models and compare some of the most useful metrics for different models and understand them thoroughly through bar graphs.****

****2.We also learned how to read the data and perform various operations such as standardization and normalization.****

****3.We also worked on how to print some rows in the data, understand if there are any null values and also get a solid understanding of how the data values are spread along with their percentile values and counts.****

****4.We've learned to plot various machine learning plots that are important and also understood the data by using various plots and features.****

****5.We found that through data visualization, we could see that the rows and features of the data that we considered were typical of the actual datasets that we use in real life.****

****6.We saw a good amount of correlation between a few features that are important and learned them in detail with the help of scatterplots.****

In []: